

PHOENIX Risk Assessment API Response Schema

1. Overview

This document explains the response schema for the PHOENIX Risk Assessment API. The main purpose of this schema is to define what the API should return after receiving an input and processing it through the risk assessment flow.

In this part of the project, the focus is not on building the full AI model. The AI team is already working on the model that will take input data and return two important values: `risk_score` and `confidence_score`. Our responsibility is to define how those values will be used inside the cybersecurity risk assessment response.

The API response should not only return a basic score. It should give a complete and understandable result. This includes whether the input is legitimate, suspicious, or malicious, what threats were found, how serious the risk is, what the final risk percentage is, and what security controls should be used to reduce that risk.

The response schema will help connect the AI model output with the cybersecurity risk assessment logic. It will also make sure that the frontend, backend, AI team, and cybersecurity team all understand the same output format.

2. Purpose of the Response Schema

The purpose of this response schema is to create a clear structure for the output of the PHOENIX Risk Assessment API.

When the API receives input such as a URL, message text, hazard type, hazard severity, source, timestamp, and alert level, the system should analyse that input and return a proper risk assessment result.

The response should answer the following questions:

- Is the input legitimate, suspicious, or malicious?
- What risk score did the AI model return?
- How confident is the AI model?
- What is the likelihood of the threat?
- What is the impact if the threat is real?
- What is the final risk score?

- What is the risk percentage?
- Is the risk low, medium, high, or critical?
- What threats were matched?
- What controls should be applied?

This makes the output useful for both technical and non-technical team members.

Instead of just returning a single number, the response gives proper context behind the decision.

3. Background of the Risk Assessment Logic

The CY019 risk assessment phase is based on turning identified cybersecurity threats into measurable risk values. Earlier phases identified possible threats using entry points, attack vectors, and STRIDE categories. CY019 then takes those threats and gives them a measurable score.

The main formula used in the risk assessment is:

$$\text{Risk Score} = \text{Likelihood} \times \text{Impact}$$

Likelihood means how likely the threat is to happen.

Impact means how serious the damage would be if the threat actually happened.

This method makes the risk assessment easier to compare and prioritise. A threat that is very likely and has a serious impact should receive more attention than a threat that is rare and has a low impact.

The attached CY019 document uses this same method to calculate risk scores and classify them into different risk levels.

4. What the API Will Receive

The API will receive the input fields that are passed into the risk assessment process. These fields are related to both cybersecurity and disaster alert information.

The expected input fields are:

```
{  
  "url": "string",  
  "text": "string",  
  "timestamp": "string",  
  "hazard_type": "string",  
  "hazard_severity": "string",  
  "hazard_timestamp": "string",  
  "hazard_location": "string",  
  "hazard_status": "string",  
  "alert_level": "string",  
  "source": "string"  
}
```

Each field has a specific role in the assessment.

5. Explanation of Input Fields

URL

The url field contains the link that needs to be checked. This may be a link from a disaster alert message, a public warning, a social media post, or another source.

A fake or suspicious URL can be a strong sign of phishing, impersonation, or misinformation.

Example:

```
"url": "http://fake-bom-alert.com"
```

This type of URL may be risky because it appears to imitate a trusted disaster information source

Text

The text field contains the actual message or content that needs to be assessed.

This can include warning messages, emergency claims, instructions, or social media-style posts.

Example:

```
"text": "Emergency flood warning. Click this link now to verify your location."
```

This text may be suspicious because it uses urgent language and asks the user to click a link.

Timestamp

The timestamp field shows when the message or request was submitted.

```
"timestamp": "2026-05-16T10:30:00Z"
```

This helps track when the request was made and can be useful for logging, investigation, and audit purposes.

Hazard Type

The hazard_type field explains what kind of disaster or emergency the input is related to.

Examples:

```
"hazard_type": "flood"
```

Other possible values may include:

bushfire
flood
storm
heatwave
earthquake
cyclone

This helps the system understand the type of situation being assessed.

Hazard Severity

The hazard_severity field explains how serious the disaster event is.

Example:

```
"hazard_severity": "high"
```

Possible values may include:

low
medium
high
critical

A high-severity hazard increases the impact of the risk because false information during a serious disaster can cause panic, confusion, and safety problems.

Hazard Timestamp

The hazard_timestamp field shows when the actual disaster event or hazard update occurred.

Example:

```
"hazard_timestamp": "2026-05-16T10:00:00Z"
```

This helps compare the timing of the hazard with the submitted message.

Hazard Location

The hazard_location field shows where the disaster or alert is related to.

Example:

```
"hazard_location": "Melbourne"
```

This is important because fake alerts in a specific location can mislead people in that area.

Hazard Status

The hazard_status field explains whether the hazard is currently active, resolved, or under monitoring.

Example:

```
"hazard_status": "active"
```

Possible values may include:

active
resolved

monitoring
unknown

If the hazard is active, the risk impact is higher because people may be depending on accurate information in real time.

Alert Level

The alert_level field shows the seriousness of the alert.

Example:

```
"alert_level": "emergency"
```

Possible values may include:

advice
watch
warning
emergency

Emergency-level alerts should be treated carefully because false or manipulated emergency alerts can create serious real-world consequences.

Source

The source field shows where the information came from.

Example:

```
"source": "unknown"
```

Possible values may include:

trusted
verified
official
unknown
unverified
third-party

An unknown or unverified source increases the risk because the system cannot confirm whether the information came from a trusted authority.

6. AI Model Output

The AI model is expected to return two values:

```
{  
  "risk_score": 0.0,  
  "confidence_score": 0.0  
}
```

These two values are the starting point for the cybersecurity risk assessment response.

Risk Score

The risk_score is a value between 0 and 1.

0 means safe

1 means unsafe

Example:

```
"risk_score": 0.88
```

A score of 0.88 means the model believes the input is highly risky.

Confidence Score

The confidence_score is also a value between 0 and 1.

0 means the model is unsure

1 means the model is very confident

Example:

```
"confidence_score": 0.91
```

A confidence score of 0.91 means the model is very confident in its prediction.

7. Cybersecurity Risk Assessment Layer

After the AI model returns `risk_score` and `confidence_score`, the cybersecurity layer will use these values to create a full risk assessment response.

This layer will calculate:

- Legitimacy
- Likelihood
- Impact
- Final risk score
- Risk percentage
- Risk level
- Matched threats
- Recommended controls
- Explanation

This step is important because the AI model only gives a numerical score. The cybersecurity layer turns that score into something meaningful and useful.

For example, the model may return:

```
{  
  "risk_score": 0.88,  
  "confidence_score": 0.91  
}
```

The cybersecurity layer can then interpret it as:

- The input is malicious.
- The likelihood is very high.
- The impact is critical.
- The final risk level is Critical.
- The recommended controls include validation, source verification, JWT, RBAC, and rate limiting.

8. Risk Calculation Method

The final risk score will be calculated using this formula:

Risk Score = Likelihood × Impact

For this API response schema, likelihood and impact will use a scale from 1 to 5.

Likelihood range: 1 to 5

Impact range: 1 to 5

Final risk score range: 1 to 25

The maximum possible risk score is:

$$5 \times 5 = 25$$

This makes the risk score easy to convert into a percentage.

9. Likelihood Scale

Likelihood shows how likely it is that the input represents a real threat.

The AI model's risk_score will be converted into likelihood.

0.00 – 0.20 = Likelihood 1

0.21 – 0.40 = Likelihood 2

0.41 – 0.60 = Likelihood 3

0.61 – 0.80 = Likelihood 4

0.81 – 1.00 = Likelihood 5

Meaning:

1 = Very unlikely

2 = Unlikely

3 = Possible

4 = Likely

5 = Highly likely

Example:

"risk_score": 0.88

This becomes:

Likelihood = 5

Because the model score is close to 1, the threat is considered highly likely.

10. Impact Scale

Impact shows how serious the damage could be if the threat is real.

Impact is calculated using the disaster context and the detected threat type.

1 = Very low impact

2 = Low impact

3 = Medium impact

4 = High impact

5 = Critical impact

Impact may increase when:

The hazard severity is high.

The alert level is emergency.

The source is unknown.

The message contains phishing content.

The URL appears fake.

The information may affect public safety.

The threat could affect alert delivery.

The threat could reduce trust in the PHOENIX system.

Because PHOENIX is used in disaster-related situations, the impact of fake information can be much higher than in a normal system.

For example, a fake shopping website is risky, but a fake emergency evacuation alert is much more serious because it can affect public safety.

11. Final Risk Score

The final risk score is calculated by multiplying likelihood and impact.

Example:

Likelihood = 5

Impact = 5

Calculation:

Final Risk Score = $5 \times 5 = 25$

This means the input has the highest possible risk score.

Another example:

Likelihood = 3

Impact = 4

Calculation:

Final Risk Score = $3 \times 4 = 12$

This would be a high risk, but not the highest possible level.

12. Risk Percentage

The risk percentage makes the final risk score easier to understand.

Formula:

Risk Percentage = $\text{Final Risk Score} / 25 \times 100$

Example:

Final Risk Score = 25

Calculation:

$25 / 25 \times 100 = 100\%$

Another example:

Final Risk Score = 12

Calculation:

$12 / 25 \times 100 = 48\%$

This means the risk percentage is 48%.

13. Risk Level Classification

The API should classify the final risk score into a clear risk level.

1–5 = Low

6–10 = Medium

11–17 = High

18–25 = Critical

Explanation:

Low

Low risk means the input does not show strong signs of being harmful. It may still be logged, but it does not need urgent action.

Medium

Medium risk means there are some warning signs. The input should be reviewed or monitored.

High

High risk means the input has strong signs of being harmful. It may involve suspicious links, unverified sources, misleading text, or disaster-related misinformation.

Critical

Critical risk means the input is highly dangerous. It may involve fake emergency alerts, phishing links, unknown sources, API abuse, or content that can affect safety and trust during a disaster.

14. Legitimacy Classification

The response should also include a legitimacy result.

The possible values are:

legitimate
suspicious
malicious

The legitimacy can be decided from the AI model's risk_score.

0.00 – 0.30 = legitimate

0.31 – 0.70 = suspicious

0.71 – 1.00 = malicious

Example:

"risk_score": 0.88

Result:

legitimacy = malicious

This means the input is likely unsafe.

15. Matched Threats

The response should include the threats that were detected or matched during the assessment.

Possible matched threats include:

Fake URL
Phishing Message
API Abuse
Fake Disaster Alert
Unverified Source
Data Tampering
Privilege Abuse
Denial of Service

Each threat should include:

Threat type
Description
STRIDE category

This helps connect the response schema with the STRIDE threat model used in the project.

The CY019 document already uses STRIDE categories such as Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.

16. STRIDE Category Mapping

The matched threats can be connected to STRIDE categories.

Fake URL → Spoofing

Phishing Message → Spoofing

Fake Disaster Alert → Spoofing / Tampering

Unverified Source → Spoofing

API Abuse → Denial of Service / Elevation of Privilege

Data Tampering → Tampering

Privilege Abuse → Elevation of Privilege

Denial of Service → Denial of Service

This makes the risk assessment more structured and easier to explain in the project documentation.

17. Recommended Security Controls

The response should also include recommended controls.

These controls explain what security actions should be used to reduce the risk.

The required controls are:

JWT Authentication

RBAC

Rate Limiting

Input Validation

Other useful controls are:

Source Verification

Digital Signature Verification

Audit Logging

Content Filtering

Domain Reputation Check

Secure API Communication
Hash Verification
Access Logging

18. Control Mapping

Fake URL

Recommended controls:

URL validation
Domain reputation check
Source verification
Input validation

Reason:

A fake URL may be used to impersonate an official disaster information source. URL validation and source verification help reduce the chance of users trusting fake links.

Phishing Message

Recommended controls:

Input validation
Content filtering
User warning system
Audit logging

Reason:

A phishing message may use urgent language, fake instructions, or suspicious links to trick users. Content filtering and validation help detect these patterns.

API Abuse

Recommended controls:

JWT Authentication
RBAC

Rate limiting
Audit logging

Reason:

API abuse can happen when attackers send too many requests, misuse endpoints, or try to access functions they should not access. JWT, RBAC, and rate limiting help protect the API.

Fake Disaster Alert

Recommended controls:

Source verification
Digital signature verification
Audit logging
Input validation

Reason:

A fake disaster alert can create panic, confusion, or unsafe behaviour. Source verification and digital signatures help confirm that alerts are genuine.

Unverified Source

Recommended controls:

Source verification
Audit logging
RBAC

Reason:

If the source is unknown, the system should not fully trust the information. It should be checked before being used for decision-making.

Data Tampering

Recommended controls:

Input validation
Hash verification

Secure API communication

Audit logging

Reason:

Data tampering can change the meaning of alerts, hazard information, or risk results. Hashing and secure communication help protect data integrity.

Privilege Abuse

Recommended controls:

RBAC

JWT Authentication

Access logging

Audit logging

Reason:

Privilege abuse happens when a user or attacker gains access to functions they should not have. RBAC limits what each user can do.

Denial of Service

Recommended controls:

Rate limiting

Request throttling

Monitoring

Logging

Reason:

Denial Service attacks can stop the system from processing important disaster information. Rate limiting helps control request traffic.

19. Final Response Schema

The API should return the response in the following structure:

```
{
  "status": "success",
  "message": "Risk assessment completed successfully",
  "data": {
    "input_summary": {
      "url": "string",
      "text": "string",
      "timestamp": "string",
      "hazard_type": "string",
      "hazard_severity": "string",
      "hazard_timestamp": "string",
      "hazard_location": "string",
      "hazard_status": "string",
      "alert_level": "string",
      "source": "string"
    },
    "model_output": {
      "risk_score": 0.0,
      "confidence_score": 0.0
    },
    "risk_assessment": {
      "legitimacy": "legitimate | suspicious | malicious",
      "likelihood": 1,
      "impact": 1,
      "final_risk_score": 1,
      "risk_percentage": 4,
      "risk_level": "Low | Medium | High | Critical"
    },
    "threat_analysis": {
      "matched_threats": [
        {
          "threat_type": "Fake URL",
          "description": "The submitted URL appears suspicious or untrusted.",
          "stride_category": "Spoofing"
        }
      ]
    }
  }
}
```

```

    }
  ]
},
"recommended_controls": [
  {
    "control": "Input Validation",
    "purpose": "Validate submitted URL, text, timestamp, hazard information, and source
before processing."
  },
  {
    "control": "JWT Authentication",
    "purpose": "Ensure only authenticated users or trusted systems can access the risk
assessment API."
  },
  {
    "control": "RBAC",
    "purpose": "Restrict access to risk assessment features based on user roles and
permissions."
  },
  {
    "control": "Rate Limiting",
    "purpose": "Prevent API abuse, spam requests, brute-force attempts, and denial-of-service
attacks."
  }
],
"explanation": "The submitted input was assessed using the AI model output, likelihood,
impact, final risk score, and mapped cybersecurity controls."
}
}

```

20. Detailed Example Response

Below is a complete example of how the API response should look after assessing a risky input.

```

{
  "status": "success",
  "message": "Risk assessment completed successfully",

```

```
"data": {
  "input_summary": {
    "url": "http://fake-bom-alert.com",
    "text": "Emergency flood warning. Click this link now to verify your location.",
    "timestamp": "2026-05-16T10:30:00Z",
    "hazard_type": "flood",
    "hazard_severity": "high",
    "hazard_timestamp": "2026-05-16T10:00:00Z",
    "hazard_location": "Melbourne",
    "hazard_status": "active",
    "alert_level": "emergency",
    "source": "unknown"
  },
  "model_output": {
    "risk_score": 0.88,
    "confidence_score": 0.91
  },
  "risk_assessment": {
    "legitimacy": "malicious",
    "likelihood": 5,
    "impact": 5,
    "final_risk_score": 25,
    "risk_percentage": 100,
    "risk_level": "Critical"
  },
  "threat_analysis": {
    "matched_threats": [
      {
        "threat_type": "Fake URL",
        "description": "The URL appears to be from an untrusted domain and may be impersonating a trusted disaster information source.",
        "stride_category": "Spoofing"
      },
      {
        "threat_type": "Phishing Message",
        "description": "The message uses urgent wording and asks the user to click a link, which is commonly seen in phishing attempts.",
        "stride_category": "Spoofing"
      }
    ]
  }
}
```

```
    },
    {
      "threat_type": "Fake Disaster Alert",
      "description": "The message claims to be an emergency disaster alert but comes from an
unknown source.",
      "stride_category": "Tampering"
    },
    {
      "threat_type": "Unverified Source",
      "description": "The source is unknown, which increases the chance of misinformation or
fake alert distribution.",
      "stride_category": "Spoofing"
    }
  ]
},
"recommended_controls": [
  {
    "control": "Input Validation",
    "purpose": "Validate URLs, message text, timestamps, hazard fields, and source values
before processing."
  },
  {
    "control": "JWT Authentication",
    "purpose": "Allow only authenticated users or trusted systems to submit requests to the
API."
  },
  {
    "control": "RBAC",
    "purpose": "Restrict sensitive API actions based on user role and permission level."
  },
  {
    "control": "Rate Limiting",
    "purpose": "Reduce spam submissions, brute-force attempts, API abuse, and denial-of-
service activity."
  },
  {
    "control": "Source Verification",
    "purpose": "Check whether the submitted alert comes from a trusted and verified source."
```

```

    },
    {
      "control": "Digital Signature Verification",
      "purpose": "Verify that disaster alerts have not been forged, modified, or injected by an
attacker."
    },
    {
      "control": "Audit Logging",
      "purpose": "Record risk assessment requests and decisions for monitoring, accountability,
and investigation."
    },
    {
      "control": "Content Filtering",
      "purpose": "Detect phishing-style language, suspicious wording, and harmful message
patterns."
    }
  ],
  "explanation": "The input was classified as malicious because the AI model returned a high
risk score, the message used urgent phishing-style wording, the alert was connected to an
emergency disaster situation, and the source was unknown. The final risk score was calculated
using likelihood and impact, which resulted in a Critical risk level."
}
}

```

21. Example for a Safer Input

This is an example of a lower-risk response.

```

{
  "status": "success",
  "message": "Risk assessment completed successfully",
  "data": {
    "input_summary": {
      "url": "https://official-alerts.gov.au/flood-update",
      "text": "Flood warning update for the affected region. Follow official emergency service

```

```
instructions.",
  "timestamp": "2026-05-16T10:30:00Z",
  "hazard_type": "flood",
  "hazard_severity": "medium",
  "hazard_timestamp": "2026-05-16T10:00:00Z",
  "hazard_location": "Melbourne",
  "hazard_status": "active",
  "alert_level": "warning",
  "source": "verified"
},
"model_output": {
  "risk_score": 0.18,
  "confidence_score": 0.86
},
"risk_assessment": {
  "legitimacy": "legitimate",
  "likelihood": 1,
  "impact": 3,
  "final_risk_score": 3,
  "risk_percentage": 12,
  "risk_level": "Low"
},
"threat_analysis": {
  "matched_threats": []
},
"recommended_controls": [
  {
    "control": "Input Validation",
    "purpose": "Continue validating URL, text, timestamp, hazard details, and source before processing."
  },
  {
    "control": "Source Verification",
    "purpose": "Confirm that the alert continues to come from a verified source."
  },
  {
    "control": "Audit Logging",
    "purpose": "Keep a record of the assessment result for traceability."
  }
]
```

```
    }  
  ],  
  "explanation": "The input was classified as legitimate because the model returned a low risk  
score, the source was verified, and no clear phishing or fake alert indicators were detected. The  
final risk score remained low."  
}  
}
```

22. Error Response Schema

The API should also return a clear error response if the request is missing required fields.

Example:

```
{  
  "status": "error",  
  "message": "Missing required fields",  
  "missing_fields": [  
    "url",  
    "text",  
    "source"  
  ]  
}
```

This helps the frontend and backend teams understand exactly what is missing.

23. Validation Error Response

If the input format is wrong, the API should return a validation error.

Example:

```
{  
  "status": "error",  
  "message": "Invalid input format",  
  "errors": [  
    {  
      "field": "risk_score",
```



```
    "issue": "risk_score must be between 0 and 1"
  },
  {
    "field": "timestamp",
    "issue": "timestamp must be a valid date-time value"
  }
]
```

This keeps the API response clean and easier to debug.

24. Why This Schema Is Useful

This response schema is useful because it gives a complete result instead of only returning a basic score.

A basic response like this is not enough:

```
{
  "risk_score": 0.88
}
```

That only tells us the model thinks the input is risky. It does not explain why the input is risky, how serious it is, what type of threat it may be, or what security controls should be applied.

A better response includes:

- Legitimacy
- Risk score
- Confidence score
- Likelihood
- Impact
- Risk percentage
- Risk level
- Matched threats
- Recommended controls
- Explanation

This makes the output easier to use in the actual PHOENIX system.

25. How This Supports the PHOENIX System

PHOENIX is designed for disaster-related cyber risk assessment. In this type of system, trust and reliability are very important.

If a fake alert is accepted as real, it can mislead people.

If a phishing link is included in a disaster message, users may be tricked into clicking it.

If the API is abused, the system may become unavailable.

If the source is not verified, false information may spread.

Because of this, the response schema needs to show more than just a score. It needs to show the full risk picture.

The schema supports PHOENIX by making sure every assessment returns:

- A clear decision

- A measurable risk score

- A risk percentage

- A risk level

- Threat details

- Security controls

- A short explanation

26. What I Need to Do for This Task

For this task, I need to prepare the risk assessment API response schema.

My work includes:

- Define the API response structure.

- Include the AI model output values: `risk_score` and `confidence_score`.

- Show how the AI risk score is converted into likelihood.

- Define how impact is represented.

Calculate the final risk score using $\text{Likelihood} \times \text{Impact}$.

Calculate the risk percentage.

Categorise the result into Low, Medium, High, or Critical.

Include legitimacy as legitimate, suspicious, or malicious.

Include matched threats such as fake URLs, phishing messages, API abuse, fake disaster alerts, and unverified sources.

Map those threats to controls such as JWT, RBAC, rate limiting, input validation, source verification, digital signature verification, and audit logging.

Provide success response examples.

Provide error response examples.

Make the schema easy for the AI team, backend team, and cybersecurity team to understand.

27. Final Summary

This document defines the response schema for the PHOENIX Risk Assessment API. The schema explains what the API should return after receiving disaster and cybersecurity-related input.

The AI model will provide two values: `risk_score` and `confidence_score`. The cybersecurity layer will then use those values to calculate likelihood, impact, final risk score, risk percentage, and risk level.

The final API response will also include legitimacy, matched threats, recommended controls, and a short explanation of why the input received that result.

This response schema keeps the output clear, consistent, and useful. It also connects the AI model output with the CY019 risk assessment method, where risks are measured using likelihood and impact.

Overall, this schema makes sure that every input returns a meaningful risk assessment result instead of just a raw model score.

